

Pygame Air Hockey Collision Fix

<https://gemini.google.com/app/74d9dcfb646ed8af>

User prompt: Act as an expert Python game developer. I am working on a 2D Air Hockey game using Pygame, and I need help fixing a bug with the puck-paddle collision logic. **The Problem:** The puck does not consistently respond correctly when it collides with a paddle, particularly at higher speeds or when it approaches at an angle. The puck is either passing through the paddle (tunneling) or getting stuck inside it. **What I need you to do:** 1. Fix the collision logic so the puck reliably rebounds from the paddles at different speeds and angles. 2. Implement overlap resolution (position correction) so that if the puck and paddle overlap, the puck is pushed out along the collision normal before its velocity is reflected. This will prevent it from getting stuck. 3. Ensure the reflection angle accounts for the 2D circular collision correctly (using vector math/dot product for circle-to-circle collision). 4. Provide the fully updated and corrected code block, and briefly explain what you changed. Here is my current code handling the collisions:

```
""" collisions: puck-vs-paddle collision handling. """
def handle_paddle_collision(puck, paddle):
    """ If the puck overlaps the paddle, bounce it off.
    Returns True if a collision was handled this frame. """
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    distance = (dx ** 2 + dy ** 2) ** 0.5
    if distance < puck.radius + paddle.radius:
        puck.vx = -puck.vx
        return True
    return False
```

Response: Here is the updated collision code. The original version was naive because it only flipped the X velocity, which ignores the angle of impact, and it didn't reposition the puck, which caused it to get trapped inside the paddle's radius over multiple frames.

```
import math

def handle_paddle_collision(puck, paddle):
    """
    If the puck overlaps the paddle, resolve overlap and bounce it off
    using 2D circle-to-circle collision vector math.
    Returns True if a collision was handled this frame.
    """
    dx = puck.x - paddle.x
    dy = puck.y - paddle.y
    distance = math.hypot(dx, dy)

    min_dist = puck.radius + paddle.radius

    if distance < min_dist:
        # Prevent division by zero if centers perfectly overlap
        if distance == 0:
            nx, ny = 1, 0
        else:
            # Calculate the collision normal (unit vector)
            nx = dx / distance
            ny = dy / distance

        # 1. Overlap Resolution (Position Correction)
        # Push the puck out along the normal so they no longer intersect
        overlap = min_dist - distance
        puck.x += nx * overlap
        puck.y += ny * overlap

        # 2. Velocity Reflection using Dot Product
        # Find how much of the puck's velocity is moving directly into the paddle
        dot_product = puck.vx * nx + puck.vy * ny

        # Only apply bounce if the puck is moving towards the paddle
        # (Prevents getting stuck if the paddle overtakes the puck from behind)
        if dot_product < 0:
            # Reflection formula: v_new = v - 2 * (v · n) * n
            puck.vx -= 2 * dot_product * nx
            puck.vy -= 2 * dot_product * ny

        # Note: If your paddle has a velocity (e.g., paddle.vx, paddle.vy),
        # you can add paddle momentum to the puck here.

    return True

return False
```

What Changed:

- **math.hypot(dx, dy):** Replaced the manual distance calculation with Python's built-in hypot for slightly better performance and precision.
- **Collision Normal Vector (nx, ny):** We now calculate the exact angle/direction of the collision by normalizing the distance between the two centers. This allows the puck to deflect realistically at an angle, rather than just flipping horizontally.
- **Overlap Resolution:** Before reflecting the velocity, we calculate the overlap amount and physically shift puck.x and puck.y out of the paddle along the normal vector. This prevents the "tunneling" and "getting stuck" bugs.
- **Dot Product Check:** The dot_product < 0 condition acts as a safeguard. It ensures we only bounce the puck if it is actively travelling toward the paddle center. If they are overlapping but already moving apart, it lets them separate smoothly.

User prompt: Act as an expert Python Pygame developer. I need to implement a full match scoring system for my 2D Air Hockey game. **Task Requirements:** 1. Goal Detection & Scoring: Detect when the puck fully/partially enters the left or right goal region. Increment the appropriate player's score by 1 when a goal occurs, and immediately reset the puck and paddles back to their starting positions. 2. Score Display (HUD): Render both players' scores continuously on the screen during gameplay (e.g., at the top center/sides of the board in a clear, readable font). 3. Winning Condition: Define a target winning score (e.g., first player to 5 or 7 points). When a player reaches this score: - Freeze active gameplay. - Display a "Player X Wins!" / "Game Over" message on screen. - Provide an option or state reset (e.g., press 'R' to restart or spacebar to reset). **What I need from you:** - Show the complete, updated code or clear snippets for the updated files (`game\_engine.py`, `renderer.py`, `main.py`, etc.). - Ensure clean separation of concerns (updating scores in the engine/game state, rendering text in the renderer). Here is my current code: """

```
GameEngine: owns the puck, both
paddles, and the computer AI, and runs one frame's worth of game logic. Starter version: the puck bounces around and paddles can hit it, but there is
no scoring, no match timer, and the reset that happens after a goal is incomplete. That's what Tasks 2-4 fix/add. """
import random from game.puck
import Puck from game.paddle import Paddle from game.ai import ComputerAI from game.collisions import handle_paddle_collision from
game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM
PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5
class GameEngine:
    def __init__(self):
        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)
        self.launch_puck()
        self.player = Paddle(
            x=WIDTH * 0.15, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.computer = Paddle(
            x=WIDTH * 0.85, y=HEIGHT / 2, radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.ai = ComputerAI()
        def _launch_puck(self):
            angle_choices = [0.3, 0.6, -0.3, -0.6]
            direction = random.choice([-1, 1])
            vy_factor = random.choice(angle_choices)
            self.puck.vx = INITIAL_PUCK_SPEED * direction
            self.puck.vy = INITIAL_PUCK_SPEED * vy_factor
        def handle_input(self, keys_pressed):
            import pygame
            dx = dy = 0
            if keys_pressed[pygame.K_UP]:
                dy -= PLAYER_SPEED
            if keys_pressed[pygame.K_DOWN]:
                dy += PLAYER_SPEED
            if keys_pressed[pygame.K_LEFT]:
                dx -= PLAYER_SPEED
            if keys_pressed[pygame.K_RIGHT]:
                dx += PLAYER_SPEED
        self.player.move_by(dx, dy)
        def update(self):
            self.ai.update(self.computer, self.puck)
            self.puck.move()
            self.puck.bounce_off_walls(HEIGHT, MARGIN)
            handle_paddle_collision(self.puck, self.player)
            handle_paddle_collision(self.puck, self.computer)
            self._handle_goals()
        def _handle_goals(self):
            if self.puck.x - self.puck.radius < MARGIN:
                if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
                    self._reset_puck()
            else:
                self.puck.x = MARGIN + self.puck.radius
                self.puck.vx = -self.puck.vx
            elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
                if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
                    self._reset_puck()
            else:
                self.puck.x = WIDTH - MARGIN - self.puck.radius
                self.puck.vx = -self.puck.vx
        def _reset_puck(self):
            self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
            self.puck.vx = 0
            self.puck.vy = 0
        def draw(self, surface, font):
            from game import renderer
            renderer.draw_table(surface)
            renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
            renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
            renderer.draw_puck(surface, self.puck)
```

Response: Here are the complete, updated snippets to implement the scoring system, HUD, and win states while maintaining a clean separation of concerns.

1. game_engine.py

The GameEngine class now manages the match state (scores, winning condition, and restarting). Game logic pauses when a win is triggered, and `_reset_board()` resets both the puck and the paddles.

```
import random
import pygame

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5
WINNING_SCORE = 5 # First to 5 wins

class GameEngine:
    def __init__(self):
        # Match State
        self.player_score = 0
        self.computer_score = 0
        self.game_over = False
        self.winner_text = ""

        # Initial positions for reset
        self.player_start_pos = (WIDTH * 0.15, HEIGHT / 2)
        self.computer_start_pos = (WIDTH * 0.85, HEIGHT / 2)

        self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)

        self.player = Paddle(
            x=self.player_start_pos[0], y=self.player_start_pos[1], radius=PADDLE_RADIUS,
            min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
            min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
        )
        self.computer = Paddle(
            x=self.computer_start_pos[0], y=self.computer_start_pos[1], radius=PADDLE_RADIUS,
            min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
```

```

        min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
    )
    self.ai = ComputerAI()
    self._launch_puck()

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    # Handle game reset if match is over
    if self.game_over:
        if keys_pressed[pygame.K_r]:
            self._reset_match()
        return # Freeze paddle movement during game over

    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self):
    if self.game_over:
        return # Freeze game logic

    self.ai.update(self.computer, self.puck)

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)

    self._handle_goals()

def _handle_goals(self):
    # Left goal (Computer scores)
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._check_win_condition()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx

    # Right goal (Player scores)
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.player_score += 1
            self._check_win_condition()
        else:
            self.puck.x = WIDTH - MARGIN - self.puck.radius
            self.puck.vx = -self.puck.vx

def _check_win_condition(self):
    if self.player_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Player Wins! Press 'R' to Restart"
    elif self.computer_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Computer Wins! Press 'R' to Restart"
    else:
        self._reset_board()

def _reset_board(self):
    """Resets puck and paddles after a goal."""
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self._launch_puck()
    self.player.x, self.player.y = self.player_start_pos
    self.computer.x, self.computer.y = self.computer_start_pos

def _reset_match(self):
    """Resets the entire game after a win."""

```

```

        self.player_score = 0
        self.computer_score = 0
        self.game_over = False
        self.winner_text = ""
        self._reset_board()

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_table(surface)
        renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
        renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
        renderer.draw_puck(surface, self.puck)

        # New rendering elements
        renderer.draw_hud(surface, font, self.player_score, self.computer_score)
        if self.game_over:
            renderer.draw_game_over(surface, font, self.winner_text)

```

2. renderer.py additions

Add these functions to your `renderer.py` module to handle the text rendering without polluting the engine logic with Pygame surface blits.

```

import pygame

# Assuming existing constants are here: WIDTH, HEIGHT, COLOR_PLAYER, etc.
COLOR_TEXT = (255, 255, 255)
COLOR_OVERLAY = (0, 0, 0, 150) # Semi-transparent black

def draw_hud(surface, font, player_score, computer_score):
    """Draws the current score at the top center of the board."""
    # Player Score
    p_text = font.render(str(player_score), True, COLOR_TEXT)
    p_rect = p_text.get_rect(center=(WIDTH * 0.25, 40))
    surface.blit(p_text, p_rect)

    # Computer Score
    c_text = font.render(str(computer_score), True, COLOR_TEXT)
    c_rect = c_text.get_rect(center=(WIDTH * 0.75, 40))
    surface.blit(c_text, c_rect)

    # Center Divider / Hyphen
    div_text = font.render("-", True, COLOR_TEXT)
    div_rect = div_text.get_rect(center=(WIDTH / 2, 40))
    surface.blit(div_text, div_rect)

def draw_game_over(surface, font, winner_text):
    """Draws a darkened overlay and the winning message."""
    # Darken screen
    overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    overlay.fill(COLOR_OVERLAY)
    surface.blit(overlay, (0, 0))

    # Draw Game Over Text
    text_surf = font.render(winner_text, True, COLOR_TEXT)
    text_rect = text_surf.get_rect(center=(WIDTH / 2, HEIGHT / 2))
    surface.blit(text_surf, text_rect)

```

Summary of Changes

- **Game State Tracking:** Added `self.player_score`, `self.computer_score`, and `self.game_over` to `GameEngine.__init__`.
- **Goal Handling:** Modified `_handle_goals` to increment the correct score when the puck falls within the `GOAL_TOP` and `GOAL_BOTTOM` thresholds.
- **Full Board Reset:** Renamed `_reset_puck` to `_reset_board` and expanded it to reset both the puck and the paddles (`self.player.x`, `self.player.y` = `self.player_start_pos`).
- **Winning Logic:** Added `_check_win_condition` which sets `self.game_over` = `True` if a player hits `WINNING_SCORE`.
- **Input / Update Freezing:** Wrapped `update()` and paddle movement inside `handle_input()` with `if not self.game_over` checks so the game pauses. The 'R' key resets the match state.
- **Renderer Decoupling:** Added `draw_hud` and `draw_game_over` to `renderer.py` to keep text rendering logic entirely out of the math/physics engine.

User prompt: Act as an expert Python Pygame developer. I need to implement a 30-second match timer with draw condition logic for my 2D Air Hockey game. ****Task Requirements:**** 1. Countdown Timer Logic ('game_engine.py'): - Add a 30-second match timer that counts down from 30.0 seconds during active gameplay (using frame delta time 'dt' or 'pygame.time.get_ticks()'). - Pause/stop the timer when 'game_over' is True. - Reset the timer

back to 30 seconds when the game is restarted (e.g., pressing 'R'). 2. Timer Completion & Outcome Logic: - When the timer reaches 0: - Immediately stop active gameplay ('game_over = True'). - Determine the match result: * If Player score > Opponent score -> Player Wins! * If Opponent score > Player score -> Computer/Opponent Wins! * If scores are equal -> Display "Match Draw!" / "Tie Game!" 3. Timer HUD Display ('renderer.py'): - Update 'draw_hud' to render the remaining time prominently on the screen (e.g., formatted as "Time: 30s" or "00:30" at the top center). **What I need from you:** - Complete updated code or snippets for 'game_engine.py', 'renderer.py', and 'main.py'. - Ensure timer updates strictly follow Pygame frame delta timing so it runs consistently across different framerates.

Response: 1. main.py

To calculate the frame delta time accurately, capture the return value of `clock.tick(60)`. By calling it at the beginning of the frame and dividing by 1000.0, we pass the exact seconds elapsed (dt) to the engine.

```
import pygame

from game.game_engine import GameEngine
from game.renderer import WINDOW_SIZE

def main():
    pygame.init()
    screen = pygame.display.set_mode(WINDOW_SIZE)
    pygame.display.set_caption("Air Hockey")
    clock = pygame.time.Clock()
    font = pygame.font.SysFont("consolas", 24)

    engine = GameEngine()
    running = True
    while running:
        # 1. Capture delta time (dt) in seconds and enforce 60 FPS
        dt = clock.tick(60) / 1000.0

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False

        keys = pygame.key.get_pressed()
        engine.handle_input(keys)

        # 2. Pass dt to the engine for timer calculations
        engine.update(dt)

        engine.draw(screen, font)
        pygame.display.flip()

    pygame.quit()

if __name__ == "__main__":
    main()
```

2. game_engine.py

The match state needs a time tracker added to the `__init__` method and a countdown added to the `update()` loop. When time runs out, gameplay freezes and the scores are compared.

```
import random
import pygame

from game.puck import Puck
from game.paddle import Paddle
from game.ai import ComputerAI
from game.collisions import handle_paddle_collision
from game.renderer import WIDTH, HEIGHT, MARGIN, GOAL_TOP, GOAL_BOTTOM

PLAYER_SPEED = 6
PUCK_RADIUS = 12
PADDLE_RADIUS = 28
INITIAL_PUCK_SPEED = 4.5
WINNING_SCORE = 5
MATCH_TIME_SECONDS = 30.0 # Added match timer constant

class GameEngine:
    def __init__(self):
        self.player_score = 0
        self.computer_score = 0
        self.game_over = False
        self.winner_text = ""
        self.time_left = MATCH_TIME_SECONDS # Initialize match time

        self.player_start_pos = (WIDTH * 0.15, HEIGHT / 2)
```

```

self.computer_start_pos = (WIDTH * 0.85, HEIGHT / 2)

self.puck = Puck(WIDTH / 2, HEIGHT / 2, PUCK_RADIUS)

self.player = Paddle(
    x=self.player_start_pos[0], y=self.player_start_pos[1], radius=PADDLE_RADIUS,
    min_x=MARGIN + PADDLE_RADIUS, max_x=WIDTH / 2 - PADDLE_RADIUS,
    min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
)
self.computer = Paddle(
    x=self.computer_start_pos[0], y=self.computer_start_pos[1], radius=PADDLE_RADIUS,
    min_x=WIDTH / 2 + PADDLE_RADIUS, max_x=WIDTH - MARGIN - PADDLE_RADIUS,
    min_y=MARGIN + PADDLE_RADIUS, max_y=HEIGHT - MARGIN - PADDLE_RADIUS,
)
self.ai = ComputerAI()
self._launch_puck()

def _launch_puck(self):
    angle_choices = [0.3, 0.6, -0.3, -0.6]
    direction = random.choice([-1, 1])
    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def handle_input(self, keys_pressed):
    if self.game_over:
        if keys_pressed[pygame.K_r]:
            self._reset_match()
        return

    dx = dy = 0
    if keys_pressed[pygame.K_UP]:
        dy -= PLAYER_SPEED
    if keys_pressed[pygame.K_DOWN]:
        dy += PLAYER_SPEED
    if keys_pressed[pygame.K_LEFT]:
        dx -= PLAYER_SPEED
    if keys_pressed[pygame.K_RIGHT]:
        dx += PLAYER_SPEED
    self.player.move_by(dx, dy)

def update(self, dt):
    if self.game_over:
        return

    # Timer countdown logic
    self.time_left -= dt
    if self.time_left <= 0:
        self.time_left = 0
        self._end_match_by_time()
        return # Skip physics updates for this frame

    self.ai.update(self.computer, self.puck)

    self.puck.move()
    self.puck.bounce_off_walls(HEIGHT, MARGIN)

    handle_paddle_collision(self.puck, self.player)
    handle_paddle_collision(self.puck, self.computer)

    self._handle_goals()

def _end_match_by_time(self):
    """Triggered when the 30-second timer hits 0."""
    self.game_over = True
    if self.player_score > self.computer_score:
        self.winner_text = "Time Up! Player Wins! Press 'R' to Restart"
    elif self.computer_score > self.player_score:
        self.winner_text = "Time Up! Computer Wins! Press 'R' to Restart"
    else:
        self.winner_text = "Time Up! Match Draw! Press 'R' to Restart"

def _handle_goals(self):
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._check_win_condition()
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx

    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:

```

```

        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.player_score += 1
            self._check_win_condition()
        else:
            self.puck.x = WIDTH - MARGIN - self.puck.radius
            self.puck.vx = -self.puck.vx

def _check_win_condition(self):
    """Retained for early wins if a player reaches WINNING_SCORE before time is up."""
    if self.player_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Player Wins! Press 'R' to Restart"
    elif self.computer_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Computer Wins! Press 'R' to Restart"
    else:
        self._reset_board()

def _reset_board(self):
    self.puck.x, self.puck.y = WIDTH / 2, HEIGHT / 2
    self._launch_puck()
    self.player.x, self.player.y = self.player_start_pos
    self.computer.x, self.computer.y = self.computer_start_pos

def _reset_match(self):
    self.player_score = 0
    self.computer_score = 0
    self.game_over = False
    self.winner_text = ""
    self.time_left = MATCH_TIME_SECONDS # Reset timer on match restart
    self._reset_board()

def draw(self, surface, font):
    from game import renderer
    renderer.draw_table(surface)
    renderer.draw_paddle(surface, self.player, renderer.COLOR_PLAYER)
    renderer.draw_paddle(surface, self.computer, renderer.COLOR_COMPUTER)
    renderer.draw_puck(surface, self.puck)

    # Pass time_left to the renderer
    renderer.draw_hud(surface, font, self.player_score, self.computer_score, self.time_left)
    if self.game_over:
        renderer.draw_game_over(surface, font, self.winner_text)

```

3. renderer.py

Update draw_hud to accept the new time_left variable and format it clearly at the top of the screen.

```

import pygame

WIDTH, HEIGHT = 800, 500
COLOR_TEXT = (255, 255, 255)
COLOR_OVERLAY = (0, 0, 0, 150)

def draw_hud(surface, font, player_score, computer_score, time_left):
    """Draws the current score and match timer at the top center of the board."""
    # Player Score
    p_text = font.render(str(player_score), True, COLOR_TEXT)
    p_rect = p_text.get_rect(center=(WIDTH * 0.25, 40))
    surface.blit(p_text, p_rect)

    # Computer Score
    c_text = font.render(str(computer_score), True, COLOR_TEXT)
    c_rect = c_text.get_rect(center=(WIDTH * 0.75, 40))
    surface.blit(c_text, c_rect)

    # Timer Display
    time_str = f"Time: {int(time_left):02d}s"
    t_text = font.render(time_str, True, COLOR_TEXT)
    t_rect = t_text.get_rect(center=(WIDTH / 2, 25))
    surface.blit(t_text, t_rect)

    # Center Divider / Hyphen
    div_text = font.render("-", True, COLOR_TEXT)
    div_rect = div_text.get_rect(center=(WIDTH / 2, 55))
    surface.blit(div_text, div_rect)

def draw_game_over(surface, font, winner_text):
    """Draws a darkened overlay and the winning message."""
    overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    overlay.fill(COLOR_OVERLAY)
    surface.blit(overlay, (0, 0))

```

```

text_surf = font.render(winner_text, True, COLOR_TEXT)
text_rect = text_surf.get_rect(center=(WIDTH / 2, HEIGHT / 2))
surface.blit(text_surf, text_rect)

```

User prompt: are these the full files that i can copy paste ??

Response: main.py and game_engine.py are complete files that you can copy and paste directly. However, the renderer.py snippet provided earlier only contained the new HUD and Game Over functions, omitting your existing table and paddle drawing logic.

Here is the fully merged renderer.py file containing both your original graphics rendering and the new timer/score overlays so you can replace the entire file.

```

"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 800, 500
MARGIN = 20
GOAL_HEIGHT = 150
GOAL_TOP = HEIGHT / 2 - GOAL_HEIGHT / 2
GOAL_BOTTOM = HEIGHT / 2 + GOAL_HEIGHT / 2

COLOR_BG = (15, 15, 25)
COLOR_TABLE = (20, 60, 90)
COLOR_WALL = (200, 200, 210)
COLOR_CENTER_LINE = (90, 130, 150)
COLOR_PUCK = (240, 240, 240)
COLOR_PLAYER = (60, 140, 240)
COLOR_COMPUTER = (240, 80, 80)
COLOR_TEXT = (255, 255, 255)
COLOR_OVERLAY = (0, 0, 0, 150)

WINDOW_SIZE = (WIDTH, HEIGHT)

def draw_table(surface):
    surface.fill(COLOR_BG)
    pygame.draw.rect(surface, COLOR_TABLE, (MARGIN, MARGIN, WIDTH - 2 * MARGIN, HEIGHT - 2 * MARGIN))
    pygame.draw.line(surface, COLOR_CENTER_LINE, (WIDTH / 2, MARGIN), (WIDTH / 2, HEIGHT - MARGIN), 2)

    pygame.draw.rect(surface, COLOR_WALL, (0, 0, WIDTH, MARGIN))
    pygame.draw.rect(surface, COLOR_WALL, (0, HEIGHT - MARGIN, WIDTH, MARGIN))
    pygame.draw.rect(surface, COLOR_WALL, (0, MARGIN, MARGIN, GOAL_TOP - MARGIN))
    pygame.draw.rect(surface, COLOR_WALL, (0, GOAL_BOTTOM, MARGIN, HEIGHT - MARGIN - GOAL_BOTTOM))
    pygame.draw.rect(surface, COLOR_WALL, (WIDTH - MARGIN, MARGIN, MARGIN, GOAL_TOP - MARGIN))
    pygame.draw.rect(surface, COLOR_WALL, (WIDTH - MARGIN, GOAL_BOTTOM, MARGIN, HEIGHT - MARGIN - GOAL_BOTTOM))

def draw_puck(surface, puck):
    pygame.draw.circle(surface, COLOR_PUCK, (int(puck.x), int(puck.y)), puck.radius)

def draw_paddle(surface, paddle, color):
    pygame.draw.circle(surface, color, (int(paddle.x), int(paddle.y)), int(paddle.radius))

def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
    surface.blit(surf, rect)

def draw_hud(surface, font, player_score, computer_score, time_left):
    """Draws the current score and match timer at the top center of the board."""
    # Player Score
    p_text = font.render(str(player_score), True, COLOR_TEXT)
    p_rect = p_text.get_rect(center=(WIDTH * 0.25, 40))
    surface.blit(p_text, p_rect)

    # Computer Score
    c_text = font.render(str(computer_score), True, COLOR_TEXT)
    c_rect = c_text.get_rect(center=(WIDTH * 0.75, 40))
    surface.blit(c_text, c_rect)

```



```

# Timer Display
time_str = f"Time: {int(time_left):02d}s"
t_text = font.render(time_str, True, COLOR_TEXT)
t_rect = t_text.get_rect(center=(WIDTH / 2, 25))
surface.blit(t_text, t_rect)

# Center Divider / Hyphen
div_text = font.render("-", True, COLOR_TEXT)
div_rect = div_text.get_rect(center=(WIDTH / 2, 55))
surface.blit(div_text, div_rect)

def draw_game_over(surface, font, winner_text):
    """Draws a darkened overlay and the winning message."""
    overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    overlay.fill(COLOR_OVERLAY)
    surface.blit(overlay, (0, 0))

    text_surf = font.render(winner_text, True, COLOR_TEXT)
    text_rect = text_surf.get_rect(center=(WIDTH / 2, HEIGHT / 2))
    surface.blit(text_surf, text_rect)

```

User prompt: Act as an expert Python Pygame developer. I need to refine and implement a clean puck and board reset sequence whenever a goal is scored in my 2D Air Hockey game. **Task Requirements:** 1. Puck Reset ('puck.py' / 'game_engine.py'): - Immediately reposition the puck to the exact center of the table (WIDTH / 2, HEIGHT / 2). - Completely reset puck velocity (vx = 0, vy = 0) and clear any forces or lingering collision state flags. - Optionally give the puck a minor initial serve velocity toward the player who was scored against, or leave it stationary until hit/served. 2. Paddle & State Reset ('paddle.py' / 'game_engine.py'): - Reset both player and computer paddles back to their default starting positions on their respective sides of the board. - Prevent paddles or puck from spawning inside each other or overlapping. 3. Seamless Play Transition: - Ensure the timer and gameplay resume smoothly for the next point without triggering accidental repeated goal detections. **What I need from you:** - Complete updated code or methods for resetting the puck and game board state cleanly. - Ensure clear separation of concerns between 'Puck.reset()', 'Paddle.reset()', and 'GameEngine._reset_board()'. Here is my current code updated:

Response: 1. puck.py

To encapsulate the puck's state management, add a reset() method to the Puck class so the engine does not have to manually manipulate the puck's coordinates and velocity properties.

```

"""
Puck: the disc players are trying to hit into the opponent's goal.
"""

class Puck:
    def __init__(self, x, y, radius=12):
        self.x = x
        self.y = y
        self.radius = radius
        self.vx = 0.0
        self.vy = 0.0

    def move(self):
        self.x += self.vx
        self.y += self.vy

    def bounce_off_walls(self, height, margin):
        """Bounce off the top and bottom walls only - left/right are handled
        separately by the game engine, since they contain the goals."""
        if self.y - self.radius < margin:
            self.y = margin + self.radius
            self.vy = -self.vy
        elif self.y + self.radius > height - margin:
            self.y = height - margin - self.radius
            self.vy = -self.vy

    def reset(self, center_x, center_y):
        """Places the puck at the center and clears all velocity/forces."""
        self.x = center_x
        self.y = center_y
        self.vx = 0.0
        self.vy = 0.0

```

2. paddle.py

To achieve clean separation of concerns, store the starting coordinates during __init__. This allows the Paddle class to own its own resetting logic rather than forcing the GameEngine to track and overwrite the paddle positions.

```

"""
Paddle: a player's or the computer's mallet, confined to their own half

```

of the table.

"""

```
class Paddle:
    def __init__(self, x, y, radius, min_x, max_x, min_y, max_y):
        self.start_x = x
        self.start_y = y
        self.x = x
        self.y = y
        self.radius = radius
        self.min_x = min_x
        self.max_x = max_x
        self.min_y = min_y
        self.max_y = max_y

    def clamp(self):
        self.x = max(self.min_x, min(self.max_x, self.x))
        self.y = max(self.min_y, min(self.max_y, self.y))

    def move_by(self, dx, dy):
        self.x += dx
        self.y += dy
        self.clamp()

    def move_to(self, x, y):
        self.x = x
        self.y = y
        self.clamp()

    def reset(self):
        """Returns the paddle to its initial starting position."""
        self.x = self.start_x
        self.y = self.start_y
        self.clamp()
```

3. game_engine.py (Targeted Updates)

Update the GameEngine methods handling the reset sequence and the goals. By passing a serve_direction parameter, the puck will automatically launch toward the player who just conceded a goal.

```
def _launch_puck(self, serve_direction=None):
    """Launches the puck. If serve_direction is specified, routes it toward that side."""
    angle_choices = [0.3, 0.6, -0.3, -0.6]

    # If no direction specified (e.g. game start), pick randomly[cite: 8]
    if serve_direction is None:
        direction = random.choice([-1, 1])
    else:
        direction = serve_direction

    vy_factor = random.choice(angle_choices)
    self.puck.vx = INITIAL_PUCK_SPEED * direction
    self.puck.vy = INITIAL_PUCK_SPEED * vy_factor

def _handle_goals(self):
    # Left goal (Computer scores)
    if self.puck.x - self.puck.radius < MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.computer_score += 1
            self._check_win_condition(serve_direction=-1) # Serve to left (Player)
        else:
            self.puck.x = MARGIN + self.puck.radius
            self.puck.vx = -self.puck.vx

    # Right goal (Player scores)
    elif self.puck.x + self.puck.radius > WIDTH - MARGIN:
        if GOAL_TOP < self.puck.y < GOAL_BOTTOM:
            self.player_score += 1
            self._check_win_condition(serve_direction=1) # Serve to right (Computer)
        else:
            self.puck.x = WIDTH - MARGIN - self.puck.radius
            self.puck.vx = -self.puck.vx

def _check_win_condition(self, serve_direction=None):
    if self.player_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Player Wins! Press 'R' to Restart"
    elif self.computer_score >= WINNING_SCORE:
        self.game_over = True
        self.winner_text = "Computer Wins! Press 'R' to Restart"
    else:
        self._reset_board(serve_direction)
```

```
def _reset_board(self, serve_direction=None):
    """Cleans board state entirely using class-level resets."""
    # Reset puck to exact center and clear velocity[cite: 8, 10]
    self.puck.reset(WIDTH / 2, HEIGHT / 2)

    # Reset paddles to their starting halves, ensuring no overlap[cite: 8, 9]
    self.player.reset()
    self.computer.reset()

    # Launch the puck to the player who conceded
    self._launch_puck(serve_direction)

def _reset_match(self):
    """Resets the entire game after a win."""
    self.player_score = 0
    self.computer_score = 0
    self.game_over = False
    self.winner_text = ""
    self.time_left = MATCH_TIME_SECONDS
    self._reset_board(serve_direction=None) # Random serve on entirely new match
```
